

Advancing Near-Future Performance Prediction in Cloud Systems Using Deep Learning

I. SYSTEM ARCHITECTURE

This section describes the high level structure of the system used in this study describing how its components are organized, interacted and work together to achieve the study's goal.

A. Python API development

At the application layer, a Python API is developed as AWS Lambda serverless API designed to be triggered by API gateway that integrates with DynamoDB to manage data sent by the function. This API is structured to work seamlessly to return responses in JSON format for both successful operations and errors. This API is accessible to multiple clients, including web applications, mobile apps, and testing pipelines. Lambda function was configured with 128MB memory, 512MB ephemeral storage and 3secs timeout.

This API has a dual-purpose design: a functioning CRUD service for structured data and a performance simulation environment that replicates realistic system characteristics.

- Health Check Endpoint
 - "/" → Returns a simple JSON response indicating that the API is active. This is primarily used to monitor the API's availability.
- CRUD Endpoints

The API simulates a lightweight in-memory database (items =) and provides the standard Create, Read, Update, Delete (CRUD) functionality:

 - POST /items/ → Creates a new item, ensuring unique names.
 - GET /items/"name" → Retrieves an item by name, returning 404 if not found.
 - GET /items/"name" → Retrieves an item by name, returning 404 if not found.
 - PUT /items/"name" → Updates an existing item with new data.
 - DELETE /items/"name" → Deletes an item by name.

This design offers a simple yet extensible data interaction model, suitable for experimentation and integration with downstream services.

- Performance Testing Endpoints

To evaluate system resilience and simulate different conditions, the API provides specialized endpoints:

 - /fast → Returns an immediate response, representing minimal latency.
 - /slow → Introduces a 2-second delay before responding, simulating a slow endpoint.
 - /random → Introduces a random delay (0.1–3.0s) to emulate unpredictable response times.
 - /unstable → Has a 20% probability of failure (returns HTTP 500) and otherwise succeeds, modeling unstable network or server conditions.
- Deployment

The Lambda-deployed API was exposed to the internet through API Gateway posting data to a serverless DynamoDB table, since Lambda functions are stateless and can run across multiple instances. Each cold start produces a fresh execution environment with an empty in-memory dictionary. At a rate of around 50 TPS, Lambda can spawn multiple concurrent instances, each maintaining its own isolated in memory for variables. The steps mentioned below describe the end-to-end deployment workflow for hosting a Python API in serverless architecture.

 - Created a Lambda function to deploy the Python API, using the default settings. Test event with default settings was created to deploy changes made to Lambda function.

≡ [Lambda](#) > [Functions](#) > Create function ? | ↺

Create function [Info](#)

Choose one of the following options to create your function.

☒ **Author from scratch**
Start with a simple Hello World example.

☐ **Use a blueprint**
Build a Lambda application from sample code and configuration presets for common use cases.

☐ **Container image**
Select a container image to deploy for your function.

Basic information

Function name [Info](#)
Enter a name that describes the purpose of your function.

Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).

Runtime [Info](#)
Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.

↻ Last fetched 12/29/2025, 12:18:52 AM

Durable execution - new [Info](#)
Enable durable execution to simplify building resilient multi-step applications that checkpoint progress and resume after interruptions. Supports Python and Node.js runtimes. [View pricing](#) [↗](#).
☐ Enable

Architecture [Info](#)
Choose the instruction set architecture you want for your function code.
☐ arm64
☒ x86_64

Permissions [Info](#)
By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. You can customize this default role later when adding triggers.

▶ **Change default execution role**

▶ **Additional configurations**
Use additional configurations to set up networking, security, and governance for your function. These settings help secure and customize your Lambda function deployment.

[Cancel](#) [Create function](#)

Fig. 1: Creating Lambda Function

- Created an API Gateway resource and integrated it with the Lambda function to handle all request methods.

API Gateway > APIs > Create API > Create REST API

Create REST API [Info](#)

☒ **New API**
Create a new REST API.

☐ **Clone existing API**
Create a copy of an API in this AWS account.

☐ **Import API**
Import an API from an OpenAPI definition.

☐ **Example API**
Learn about API Gateway with an example API.

API name

My REST API

Description - optional

API endpoint type
Regional APIs are deployed in the current AWS Region. Edge-optimized APIs route requests to the nearest CloudFront Point of Presence. Private APIs are only accessible from VPCs.

Regional

Security policy - new [Info](#)
Transport Layer Security (TLS) protects data in transit between a client and server. The security policy also determines the cipher suite options that clients can use with your API.

SecurityPolicy_TLS13_1_2_2021_06

Endpoint access mode - new [Info](#)
Provide additional governance for your APIs.

☒ **Basic**
Allow all clients to access the API

☐ **Strict (recommended)**
Enforce Server Name Indication (SNI) validation

IP address type [Info](#)
Select the type of IP addresses that can invoke the default endpoint for your API.

☒ **IPv4**
Supports only edge-optimized and Regional API endpoint types.

☐ **Dualstack**
Supports all API endpoint types.

Cancel Create API

Fig. 2: Creating RESTAPI using APIgateway

API Gateway > APIs > Resources - Lambda_Items (bjdpxm4m6f) > Create resource

Create resource

☒ **Proxy resource** [Info](#)
Proxy resources handle requests to all sub-resources. To create a proxy resource use a path parameter that ends with a plus sign, for example {proxy+}.

Resource path

/

Resource name

/[proxy+]

☐ **CORS (Cross Origin Resource Sharing)** [Info](#)
Create an OPTIONS method that allows all origins, all methods, and several common headers.

Cancel Create resource

Fig. 3: Creating Resource for RESTAPI

- Enabled Lambda proxy integration to ensure full request and response pass through.

API Gateway > APIs > Resources - Lambda_Items (bjdpxm4m6f)

Resources

API actions Deploy API

Create resource

- /
- ANY
- /{proxy+}
- ANY

/{proxy+} - ANY - Method execution

Update documentation Delete

ARN: `arn:aws:execute-api:us-east-2:190224075906:bjdpxm4m6f/*//{proxy+}` Resource ID: `1yu1to`

Client → Method request → Integration request → Lambda integration → Integration response (Proxy integration) → Method response

Method request Integration request Integration response Method response

Integration request settings

Edit

Integration type: Lambda Region: us-east-2

Lambda proxy integration: True Response transfer mode: Buffered

Fig. 4: Enabled Lambda Proxy Integration

- Attached the policies as mentioned in the figure to allow Lambda publish logs, metrics, and insights to CloudWatch.

IAM > Roles > NewLambda-role-4l4txmya

NewLambda-role-4l4txmya

Delete Edit

Summary

Creation date: November 20, 2025, 23:56 (UTC-06:00)

Last activity: 1 month ago

ARN: `arn:aws:iam::190224075906:role/service-role/NewLambda-role-4l4txmya`

Maximum session duration: 1 hour

Permissions Trust relationships Tags Last Accessed Revoke sessions

Permissions policies (4)

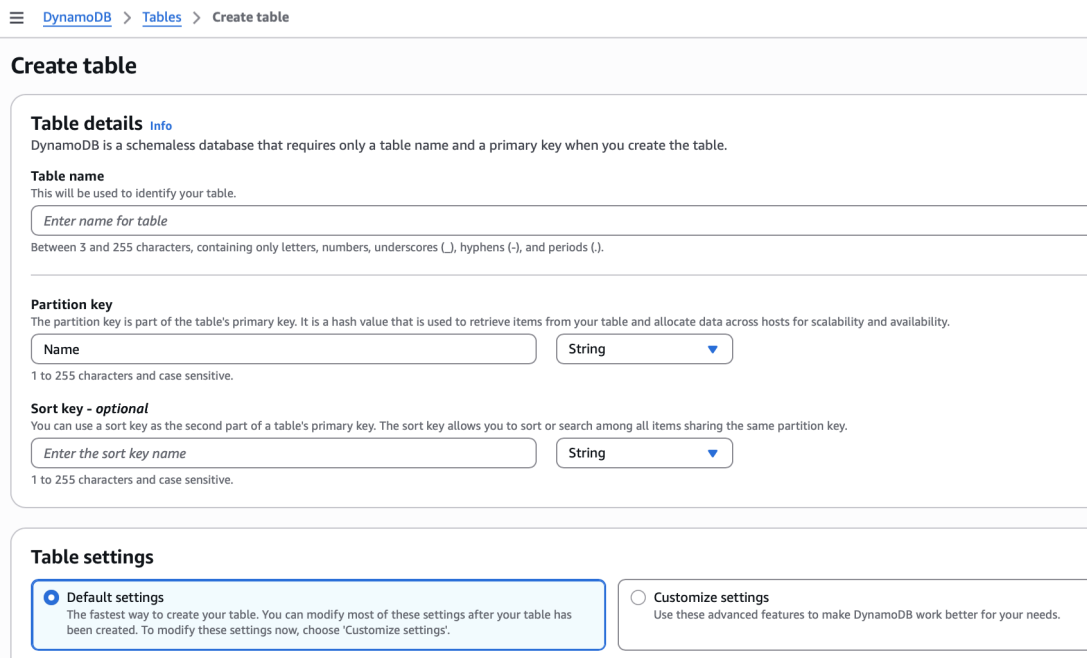
You can attach up to 10 managed policies.

Filter by Type: All types

Policy name	Type	Attached entities
AmazonDynamoDBFullAccess_v2	AWS managed	2
AWSLambdaBasicExecutionRole-f06759c6-8fac-4c6c-8253-31ea970f841e	Customer managed	1
AWSLambdaDynamoDBExecutionRole	AWS managed	1
AWSLambdaVPCEAccessExecutionRole-ed7785f6-ff3f-4b18-ac61-3b4ef3747...	Customer managed	1

Fig. 5: Policies for Lambda to Send Logs to CloudWatch

- Created a DynamoDB table with Name as primary key and with default settings. Lambda code was later updated to write incoming data to this table.



Create table

Table details [Info](#)
DynamoDB is a schemaless database that requires only a table name and a primary key when you create the table.

Table name
This will be used to identify your table.

Between 3 and 255 characters, containing only letters, numbers, underscores (_), hyphens (-), and periods (.).

Partition key
The partition key is part of the table's primary key. It is a hash value that is used to retrieve items from your table and allocate data across hosts for scalability and availability.

1 to 255 characters and case sensitive.

Sort key - optional
You can use a sort key as the second part of a table's primary key. The sort key allows you to sort or search among all items sharing the same partition key.

1 to 255 characters and case sensitive.

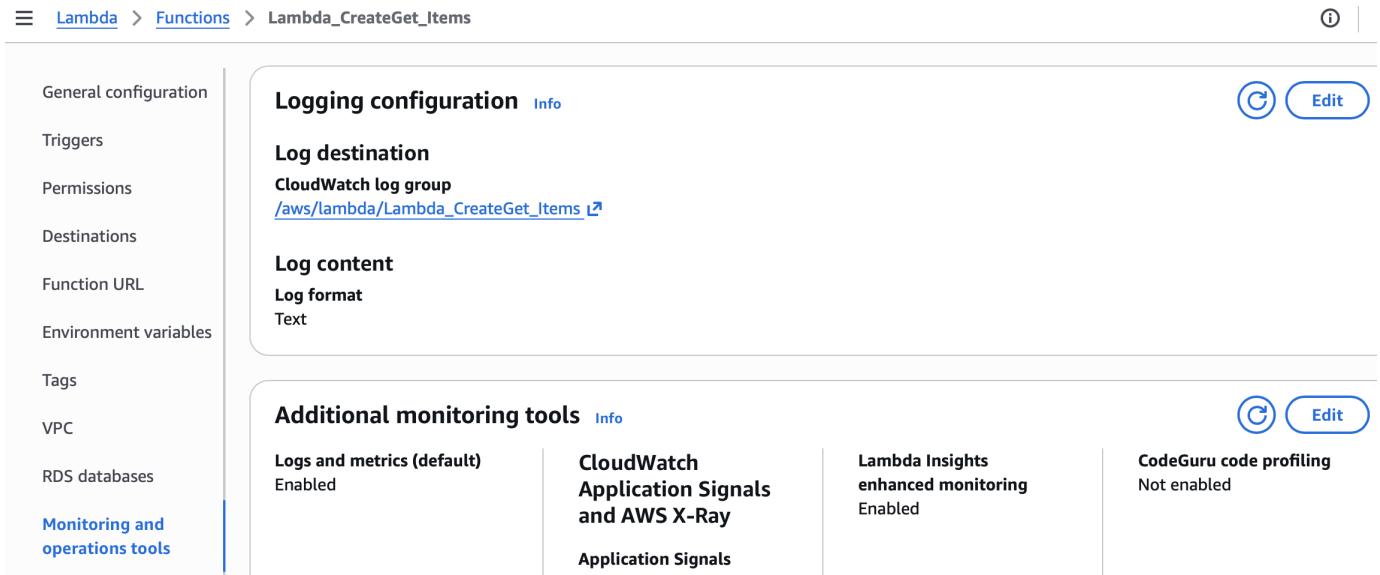
Table settings

☒ **Default settings**
The fastest way to create your table. You can modify most of these settings after your table has been created. To modify these settings now, choose 'Customize settings'.

☐ **Customize settings**
Use these advanced features to make DynamoDB work better for your needs.

Fig. 6: Table Creation in DynamoDB

- Enabled Lambda insights enhanced monitoring under lambda configuration - monitoring and operations tools.



Lambda > **Functions** > **Lambda_CreateGet_Items**

General configuration

Triggers

Permissions

Destinations

Function URL

Environment variables

Tags

VPC

RDS databases

Monitoring and operations tools

Logging configuration [Info](#)

Log destination
CloudWatch log group
[/aws/lambda/Lambda_CreateGet_Items](#)

Log content
Log format
Text

Additional monitoring tools [Info](#)

Logs and metrics (default)
Enabled

CloudWatch Application Signals and AWS X-Ray
Application Signals

Lambda Insights enhanced monitoring
Enabled

CodeGuru code profiling
Not enabled

Fig. 7: Enable Lambda Insights to Send Metrics to CloudWatch

B. Executing Tests with JMeter

This JMeter script was designed to perform CRUD operations: Add, Get, Update, and Delete items using "HTTP Request" element. Test data for the script was injected using CSV Data Set Config element with data containing the fields Name, Description, Price, Quantity, Update item, and Endpoint. Test was executed on serverless architecture to gather the data with the load slowly ramping up to 50 TPS to sustain at 50 TPS for one hour to train the models. Some of the performance issues like slower transactions and unstable transactions were induced to test the predictability of the models.

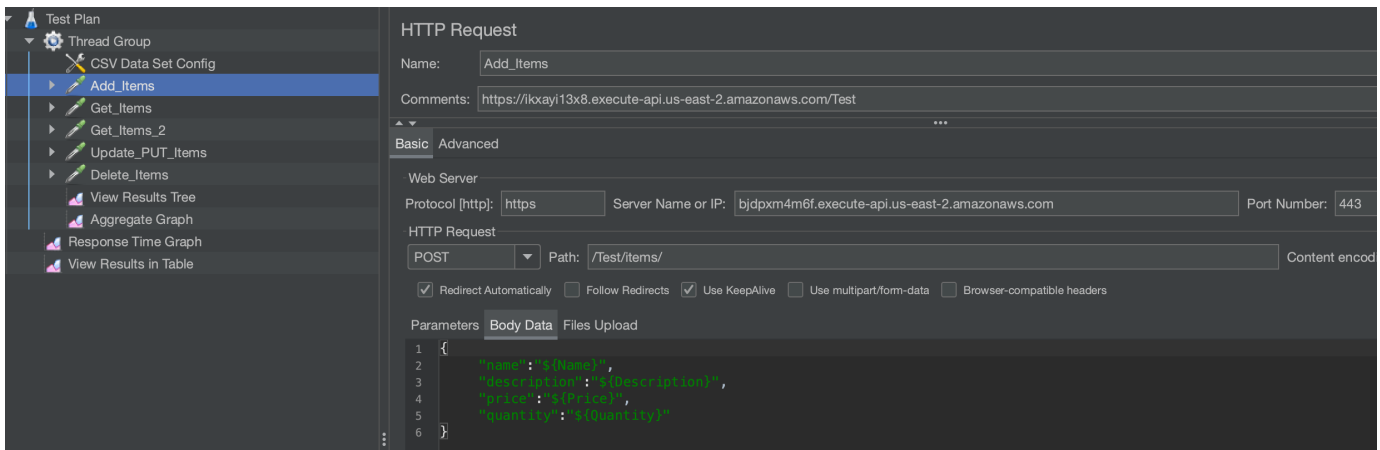


Fig. 8: Jmeter Request to Add Items Request

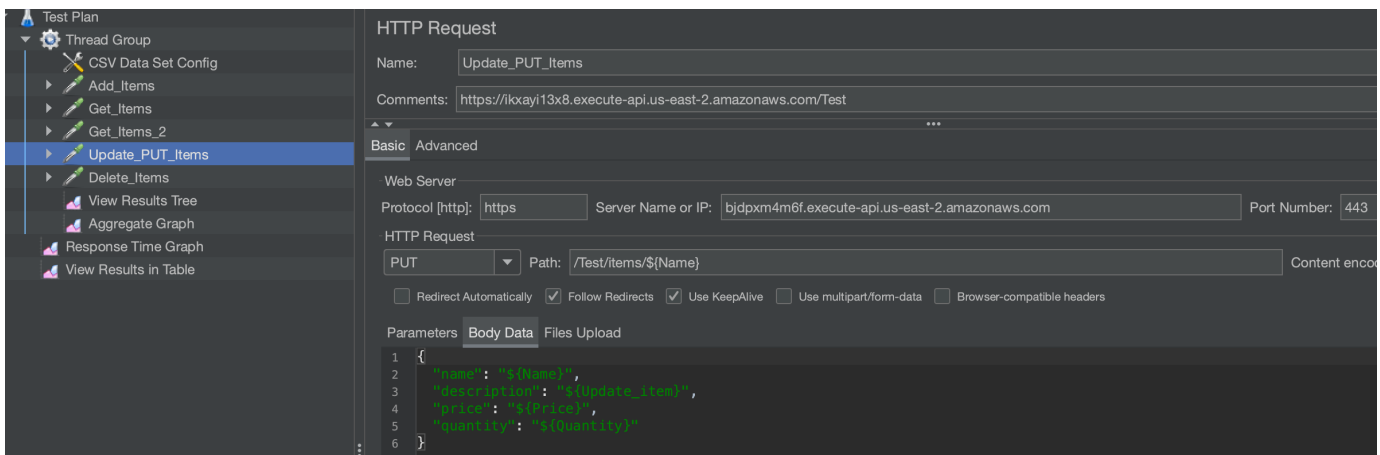


Fig. 9: Jmeter Request to Update Items Request

All other requests followed the same configuration approach with below mentioned configuration (on port 443), endpoints and empty body.

- Get Item: /Test/items/\${Name}
- Get API status with induced performance bottleneck: /Test/ \$Endpoint
- Delete an Item: /Test/items/\${Name}

C. Validating Metrics in CloudWatch

API deployed on AWS Lambda publishes its operational metrics to Amazon CloudWatch after the required IAM policies are enabled for both Lambda and API Gateway. In addition to standard metrics Enhanced Lambda Monitoring is enabled to capture detailed resource-level metrics, including CPU utilization, memory usage, disk, and network activity. These metrics can be viewed in CloudWatch under Lambda Insights, providing comprehensive visibility into the function's performance and resource behavior.

D. Retrieving Performance Metrics to SageMaker

Python monitoring and analysis script was written in SageMaker JupyterLab notebook to retrieve performance data to use for training the models. It queries AWS CloudWatch to collect key performance metrics and organizes the results into a clear, readable format.

Create notebook instance

Amazon SageMaker provides pre-built fully managed notebook instances that run Jupyter notebooks. The notebook instances include example code for common model training and hosting exercises. [Learn more](#)

Notebook instance settings

Notebook instance name

Maximum of 63 alphanumeric characters. Can include hyphens (-), but not spaces. Must be unique within your account in an AWS Region.

Notebook instance type

Platform identifier [Learn more](#)

► Additional configuration

Permissions and encryption

IAM role

Notebook instances require permissions to call other services including SageMaker and S3. Choose a role or let us create a role with the [AmazonSageMakerFullAccess](#) IAM policy attached.

[Create role using the role creation wizard](#)

Root access - optional

☒ Enable - Give users root access to the notebook

Fig. 10: JupyterLab Notebook in SageMaker AI to Train Models

E. Preparing Time-Series data

A python code was written that prepares data to use it in training machine learning model such as LSTM. It converts sequence of data points into overlapping input windows and corresponding target values, allowing the model to learn patterns from recent observations and predict the next value in the series. The final output shows how the data has been structured into inputs and labels, confirming that it is ready for training a sequence-based prediction model.

F. PyTorch LSTM Time-Series Training Loop

This python code defines and trains a simple LSTM-based neural network using PyTorch to forecast future values from time-series past data. The training loop repeatedly feeds the retrieved data from CloudWatch into the model, calculates prediction error using mean squared loss, and updates the model parameters through backpropagation and optimization. Over multiple training cycles or epochs, the model gradually improves its ability to predict time-series behavior, with the loss printed each time to track learning progress.

G. Performance Bottlenecks Predictions

Python code written in this section uses trained LSTM model to generate future predictions from time-series data obtained in above section. It repeatedly feeds a sliding window of data into the model to predict the next time step, then appends that prediction back into the input sequence. This process is repeated to forecast multiple future points ahead. Finally, the predicted values are converted back to their original scale and printed, providing an estimate of how the performance metrics may behave in the near future.